

CURSO DE C#

Capítulo 2

Tipos de datos en C#

Desde principiante hasta nivel práctico

Enfoque: Desarrollo de C#

Objetivo del capítulo

Aprenderás cómo C# representa información mediante tipos de datos, cómo elegirlos y cómo convertirlos de manera segura. Podrás aplicar estas decisiones en programas de inventario, sensores, producción, robots y aplicaciones empresariales.

Al finalizar este capítulo serás capaz de:

- Explicar qué es un tipo de dato y por qué existe.
- Elegir tipos para enteros, decimales, texto, caracteres, estados y fechas.
- Distinguir tipos por valor y por referencia a nivel introductorio.
- Comprender rangos, precisión, null y conversiones.
- Detectar desbordamientos, pérdida de precisión y conversiones inválidas.
- Justificar la elección de tipos en un proyecto C#.

1. Introducción al tema

Un tipo de dato es una clasificación que indica qué clase de información puede representar una variable y qué operaciones son válidas sobre ella. El valor 10 puede significar diez piezas, diez grados o el texto "10"; el tipo permite distinguir esos significados.

La analogía es la de un almacén con recipientes especializados: uno admite piezas enteras, otro líquidos y otro documentos. En C#, el tipo establece reglas semejantes. Sin tipos, el compilador no podría comprobar muchas operaciones y los errores aparecerían más tarde.

Los tipos se utilizan en formularios, bases de datos, videojuegos, sensores, PLC, automatización, nóminas, inventarios y aplicaciones web. Se relacionan con variables, operadores, condiciones, ciclos, métodos, arreglos y clases.

2. Concepto fundamental: tipo, nombre y valor

¿Qué es?

El tipo describe la naturaleza del dato; el nombre permite acceder a él; el valor es la información representada.

¿Por qué y cuándo se utiliza?

Cada variable, parámetro, propiedad o resultado necesita un tipo para que el compilador pueda verificar operaciones y para que el programa interprete correctamente la información.

¿Cómo funciona?

C# usa tipado estático: el tipo se conoce en compilación. Una variable int no puede recibir directamente un string.

```
int piezas = 120;  
Console.WriteLine(piezas);
```

La primera línea declara e inicializa un entero. La segunda lee su valor y lo muestra. El resultado es 120.

3. Categorías principales

Categoría	Ejemplos	Uso
Enteros	byte, short, int, long	Conteos y cantidades
Punto flotante	float, double	Mediciones y cálculos aproximados
Decimal	decimal	Importes y precisión decimal
Lógico	bool	Estados y decisiones

Categoría	Ejemplos	Uso
Carácter y texto	char, string	Letras, nombres y mensajes
Tiempo	DateTime, DateOnly, TimeOnly, TimeSpan	Fechas, horas y duraciones
Definidos por usuario	class, struct, enum, record	Modelar entidades

4. Tipos enteros

Representan números sin parte fraccionaria. Se diferencian por rango y por permitir o no valores negativos.

Tipo	Uso orientativo
byte	Valores pequeños sin signo, como 0 a 255.
short/ushort	Formatos o protocolos específicos.
int	Contadores, piezas, edades y cantidades comunes.
uint	Cantidades que nunca deben ser negativas.
long/ulong	Contadores o identificadores muy grandes.

```
int piezas = 125;
long eventos = 4500000000L;
byte porcentaje = 85;
```

El sufijo L indica un literal long. No elijas byte solo porque ocupa menos: el rango y el significado importan.

5. float, double y decimal

float y double representan fracciones con precisión aproximada. decimal utiliza una representación decimal apropiada para muchos cálculos monetarios.

Tipo	Cuándo usarlo	Precaución
float	Gráficos o simulaciones donde el tamaño importa.	Menor precisión; suele usar sufijo f.
double	Sensores, ciencia y cálculos generales.	No todos los decimales son exactos.
decimal	Precios, salarios e importes.	Puede ser más pesado y no es universalmente mejor.

```
double temperatura = 23.75;
float velocidad = 12.5f;
decimal precio = 149.99m;
```

Los sufijos f y m indican float y decimal. Para dinero, decimal suele ser una elección más adecuada que double.

6. bool, char y string

```
bool activa = true;
char inicial = 'F';
string nombre = "Francisco";
```

bool solo admite true o false. char representa un carácter y usa comillas simples. string representa texto y usa comillas dobles. No confundas char letra = 'A' con char letra = "A", que es incorrecto.

7. Fechas y duraciones

Tipo	Representa
DateTime	Fecha y hora.
DateOnly	Solo fecha.
TimeOnly	Solo hora.
TimeSpan	Duración o diferencia temporal.

```
DateTime inicio = new DateTime(2026, 9, 4, 8, 0, 0);
DateTime fin = inicio.AddHours(8);
TimeSpan duracion = fin - inicio;
Console.WriteLine(duracion.TotalHours);
```

El programa crea una hora inicial, suma ocho horas y calcula una duración. Usar tipos temporales facilita comparar y operar con fechas.

8. Tipos por valor y por referencia

Los tipos por valor representan directamente su valor; al asignarlos normalmente se copia el valor. Los tipos por referencia contienen una referencia a un objeto.

```
int a = 10;
int b = a;
b = 20;
Console.WriteLine(a); // 10

int[] primero = { 1, 2 };
int[] segundo = primero;
segundo[0] = 99;
Console.WriteLine(primero[0]); // 99
```

En el primer caso, b recibe una copia. En el segundo, ambos nombres apuntan al mismo arreglo.

9. Conversiones

Una conversión cambia un valor de un tipo a otro. Puede ser implícita o explícita.

```
int piezas = 25;
double cantidad = piezas;
double lectura = 25.8;
int entero = (int)lectura;
```

La conversión explícita a int descarta la parte fraccionaria. No equivale necesariamente a redondear.

```
string entrada = "150";
if (int.TryParse(entrada, out int resultado))
{
    Console.WriteLine(resultado);
}
else
{
    Console.WriteLine("Entrada inválida");
}
```

TryParse permite comprobar si una conversión tuvo éxito. Es especialmente útil para entradas de usuario.

10. null y tipos anulables

null representa ausencia de valor. Un tipo por valor ordinario no admite null, pero puede hacerse anulable con ?. Cero es un valor; null significa que no hay dato.

```
int? codigoAlarma = null;
codigoAlarma = 12;
Console.WriteLine(codigoAlarma);
```

11. Ejemplo profesional: estación de producción

```
string codigo = "EST-04";
int piezas = 1250;
double temperatura = 68.4;
decimal costoPorPieza = 12.50m;
bool activa = true;
DateTime ultimaLectura = DateTime.Now;
int? codigoAlarma = null;

Console.WriteLine($"Estación: {codigo}");
Console.WriteLine($"Piezas: {piezas}");
Console.WriteLine($"Temperatura: {temperatura}");
Console.WriteLine($"Costo: {costoPorPieza}");
Console.WriteLine($"Activa: {activa}");
Console.WriteLine($"Alarma: {codigoAlarma}");
```

El ejemplo separa identidad, cantidades, mediciones, dinero, estados, tiempo y ausencia de información. Esta separación facilita validación, mantenimiento y futuras conexiones con sensores o bases de datos.

Errores de confusión comunes

Confusión	Diferencia
int/double	int no representa fracciones; double sí, aproximadamente.
double/decimal	Tienen distinta representación y propósito.
char/string	Uno es un carácter; el otro es texto.
0/null	Uno es un valor; el otro es ausencia.
Parse/TryParse	Parse puede fallar con excepción; TryParse informa éxito.
var/dynamic	var conserva tipo estático; dynamic aplaza comprobaciones.

Errores comunes al trabajar con tipos de datos

Elegir un rango insuficiente: puede causar desbordamiento o resultados incorrectos. Corrige analizando límites reales.

Usar string para cálculos: convierte y valida antes de operar.

Perder decimales: una conversión a int descarta la fracción; decide si truncar o redondear.

Ignorar entradas inválidas: usa TryParse cuando el dato provenga del usuario.

Guardar fechas como texto sin necesidad: usa tipos temporales para ordenar y calcular.

Confundir null con cero o texto vacío: define claramente qué significa ausencia.

Elegir tipos sin considerar el dominio: el tipo debe representar el significado del dato.

Buenas prácticas

- Elige tipos por significado, rango, precisión y operaciones.
- Usa int para conteos normales y evita tipos pequeños salvo una razón clara.

- Prefiere decimal para muchos importes.
- Usa tipos temporales para fechas y duraciones.
- Valida datos externos y considera TryParse.
- Usa nombres con unidades cuando sea útil, como temperaturaCelsius.
- Documenta decisiones poco obvias.
- Considera null explícitamente cuando un dato pueda no existir.

Conectando conocimientos

Las variables del capítulo anterior ahora se entienden con mayor profundidad: el tipo define qué valores son válidos. Los operadores dependen del tipo; las condiciones producen bool; los ciclos usan contadores; y las clases agrupan datos de distintos tipos. El siguiente capítulo podrá convertir entradas, que normalmente llegan como string, al tipo requerido.

Ejercicios prácticos

Ejercicio 1 — Comprensión básica

Define variables para nombre de empleado, edad, salario, contratado y primera letra del apellido. Elige y justifica cada tipo. Pista: clasifica texto, entero, decimal, lógico y carácter.

Ejercicio 2 — Aplicación

Una tienda registra precio, cantidad, descuento y disponibilidad. Elige tipos, calcula subtotal y muestra los datos. No uses string para cálculos.

Ejercicio 3 — Integración

Representa un sensor con código, lectura, unidad, fecha y estado. Integra variables, salida y una conversión segura.

Ejercicio 4 — Caso práctico

Diseña el modelo de datos inicial de una máquina industrial con producción, temperatura, presión, costo, mantenimiento y alarma opcional.

Ejercicio 5 — Reto

Crea un registro de turno que combine operador, producción, calidad y tiempo. Usa al menos seis tipos, una fecha, una duración, un anulable y una conversión segura.

Mini proyecto: Registro de producción tipado

Contexto

Una empresa necesita registrar información de una línea de producción y preparar sus datos para futuros capítulos.

Objetivo

Aplicar tipos correctos para identidad, cantidades, mediciones, dinero, estados, tiempo y ausencia de datos.

Requisitos funcionales

- Código y nombre de línea.
- Piezas producidas y rechazadas.
- Temperatura y presión.
- Costo por pieza.
- Estado activo.

- Fecha y hora del turno.
- Código de alarma opcional.
- Reporte final.

Requisitos técnicos

- Usar string, int, double, decimal, bool y DateTime.
- Usar un tipo anulable.
- No usar texto para cálculos.
- Comentar decisiones importantes.

Paso 1

Haz una tabla con dato, significado, tipo y razón.

Paso 2

Declara datos de identidad, producción y estado; muestra un reporte.

Paso 3

Agrega mediciones, costo y fecha; calcula costo estimado o porcentaje de rechazo.

Paso final

Integra la alarma opcional y prueba valores normales, cero, decimales y ausencia de alarma.

Comprueba tus conocimientos

1. ¿Qué problema resuelve un tipo de dato?
2. ¿Por qué int no representa 12.75?
3. ¿Qué diferencia existe entre float, double y decimal?
4. ¿Qué tipo usarías para una máquina activa?
5. ¿Qué diferencia hay entre char y string?
6. ¿Qué significa tipado estático?
7. ¿Qué sucede al convertir 15.9 a int?
8. ¿Por qué TryParse puede ser preferible a Parse?
9. ¿Qué diferencia hay entre 0 y null?
10. ¿Qué tipo representa una duración?
11. ¿Por qué "20" no equivale a 20?
12. ¿Por qué usar decimal para dinero?
13. ¿Qué sucede si dos referencias apuntan al mismo arreglo?
14. Propón tipos para código de robot, peso, emergencia y mantenimiento.
15. ¿Qué riesgo existe al elegir un rango insuficiente?

Respuestas y explicaciones

1. Define la naturaleza del valor y las operaciones válidas.
2. Porque int solo representa enteros.
3. Difieren en precisión, representación y propósito.

4. bool.
5. char es un carácter; string es una secuencia.
6. El compilador conoce y verifica tipos antes de ejecutar.
7. Se pierde la parte fraccionaria.
8. Permite comprobar éxito sin depender necesariamente de una excepción.
9. 0 es valor; null es ausencia.
10. TimeSpan.
11. El primero es texto y el segundo número.
12. Porque suele ser apropiado para precisión decimal monetaria.
13. Ambas referencias pueden modificar el mismo arreglo.
14. string, double o decimal según requisitos, bool y DateTime.
15. Puede causar desbordamiento o resultados incorrectos.

Resumen del capítulo

Concepto	Uso	Error común
int	Conteos y enteros	Usarlo para fracciones
double	Mediciones aproximadas	Esperar exactitud total
decimal	Importes	Usarlo sin necesidad
bool	Estados lógicos	Guardarlo como texto
char/string	Carácter y texto	Confundir comillas
DateTime/TimeSpan	Fechas y duraciones	Usar texto para todo
nullable	Ausencia de valor	Confundir null con cero

Ideas clave que debes recordar

- El tipo define la naturaleza y las reglas del valor.
- Elegir tipos es una decisión de diseño.
- int sirve para muchos conteos; double para mediciones; decimal para muchos importes.
- bool representa lógica; char un carácter; string texto.
- Las conversiones deben ser conscientes y seguras.
- TryParse permite validar conversiones.
- null no significa cero.
- Los tipos por valor y referencia se copian de forma diferente.
- Un buen tipo mejora claridad, seguridad y mantenimiento.

Mapa de aprendizaje

Variables → clasificación de datos → elección de tipos → conversiones → validación → operadores → decisiones y ciclos → arreglos y métodos → clases → aplicaciones profesionales.

Este capítulo prepara el estudio de entrada de usuario: los datos suelen llegar como texto y deberán transformarse al tipo correcto antes de utilizarse.